

The background features a light gray grid with thin black lines forming a circuit-like pattern. Various colored nodes (green circles, yellow hexagons, red hexagons) are placed at different points along these lines. Concentric circular arcs are also present, particularly on the right side of the image.

나가자 (NAGAJA) : 동적 알람 및 시간 관리 시스템

당신의 아침을 예측하고, 스스로 시간을 조정하는 능동형 시간의 대시보드

가드팀 캡스톤디자인 중간발표

왜 우리는 매일 같은 시간에 알람을 맞춰도 지각하는가?



기상 08:00

이동 시간 30분 고정



09:00
도착 예정



기상 08:00

폭우 발생



교통 체증



09:15 도착 확정 (지각)

기존 알람은 '개인 준비 시간'과 실시간 '환경 변수'를 방치합니다.

고정된 알람의 한계 vs 능동적 시간 관리

	기존 스마트폰 알람	나가자 시스템
알람 설정 방식	고정된 시간 수동 설정 	일정 및 이동 시간에 맞춘 자동 역산 
외부 변수 반영	반영 불가 (비가 와도 동일 시간) 	날씨 및 교통 상황에 따른 동적 시간 조정 
학습 및 보정	없음 	출결 기록 기반 개인화 오차 보정 루프 
지각 대처	알람 종료 후 방치 	실시간 남은 시간에 따른 4단계 행동 가이드 제공 

시간의 역산: 정밀한 출발 시간을 도출하는 알고리즘



단순한 '소요 시간' 안내를 넘어, 사용자가
'실제로 침대에서 일어나야 하는 시간'을 동적으로 계산하여 제시합니다.

State Machine: 숫자가 아닌 '상태'로 행동을 유도하는 UX



3-Tier 시스템 아키텍처 및 30% 구현 현황

Mobile App



사용자 인터랙션, 상태 UI,
데이터 입력.

30% 완료

Firebase Backend



데이터 중앙 처리, 시간 동적
계산, 외부 API 호출.

30% 완료

Hardware (Raspberry Pi)



물리적 피드백, 엣지 컴퓨팅
기반 단독 알람 구동.

30% 완료

The Feedback Loop: 실제 행동 데이터 기반 오차 자가 보정

Step 4 [파라미터 업데이트]

DB의 '개인 준비시간' 및
'기본 이동시간' 파라미터
자동 보정 (Self-Learning).

Step 1 [예측]

앱이 외부 API를 융합하여
동적 계산된 출발 시간 제안.

Step 3 [비교 분석]

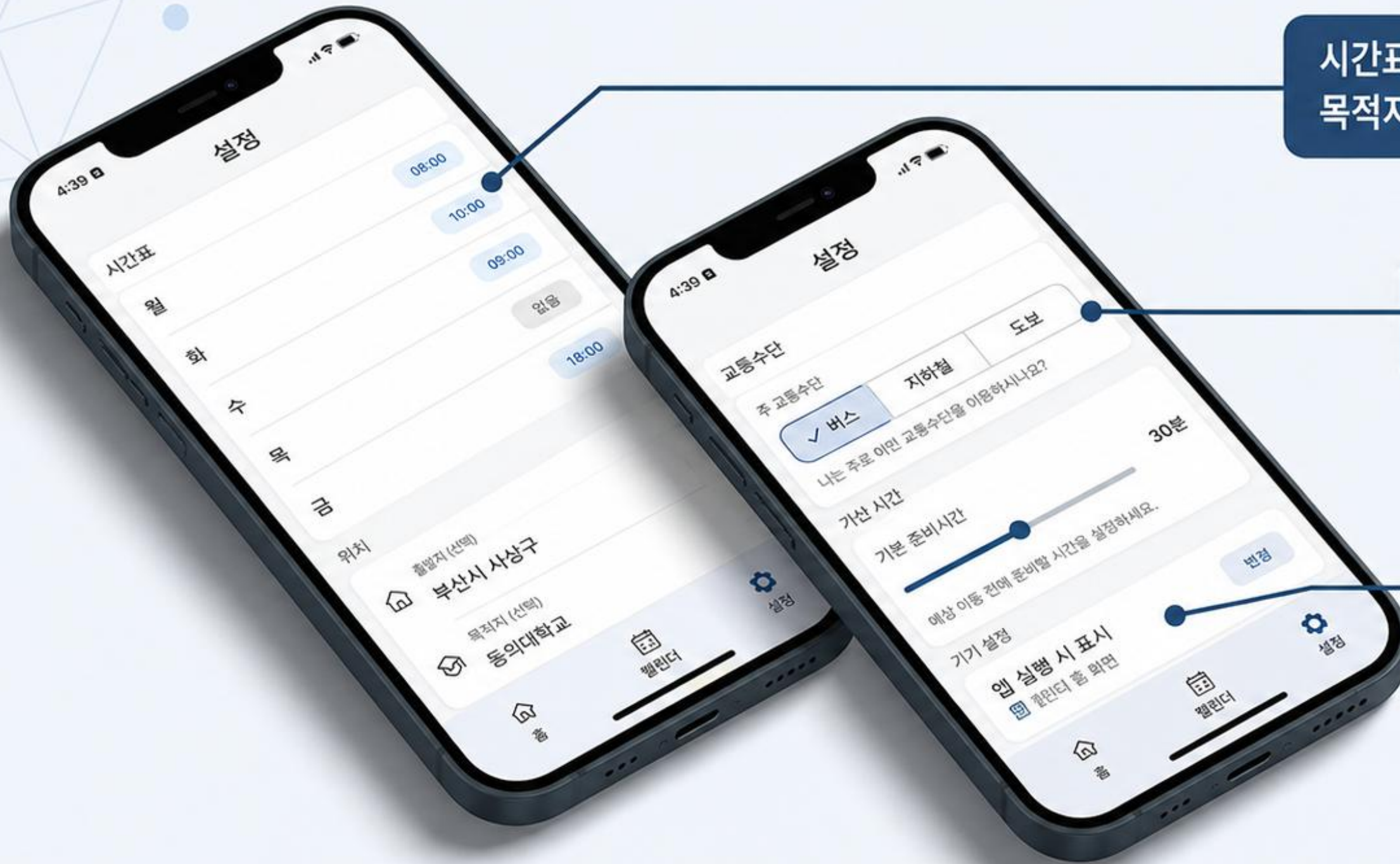
예측 이동 시간 vs 실제 이동
시간의 오차 및 지각 여부 분석.

Step 2 [기록 - Zero Effort UX]

- 집 Wi-Fi 연결 해제를 감지하여
스마트폰 조작 없이 '실제 출발
시각' 자동 기록.



Milestone 1: Data Input & Mobile UI 파이프라인 완성



시간표 및 위치: 요일별 스케줄과 출발지/목적지를 입력받아 기본 이동 경로를 산출합니다.

교통수단: 버스 선택 시 배차 대기시간을 이동 시간에 추가하여 계산 정확도를 높입니다.

개인 준비시간 (Prep Minutes): 세면, 옷 입기 등 집을 나서기까지의 개인화된 초기값을 설정합니다. (현재 30분)

데이터를 수집하여 변수(시간표, 위치, 교통수단, 준비시간)를 세팅할 수 있는 모바일 프론트엔드 기초 세팅을 완료했습니다.

Milestone 2: 확장성을 고려한 유연한 동적 DB 스키마 설계

```
{  
  "user": {  
    "name": "schedules",  
    "username": null,  
    "prepMinutes": 15,  
    "status": [],  
    "homeWifiSsids": ["HOME_WIFI"],  
    ...  
  }  
}
```

`Users` (마스터 데이터)

사용자 식별, Wi-Fi 기반
위치 트리거
(`homeWifiSsids`), 개인
기준값(`prepMinutes`)
저장.

`Schedules` (정적 템플릿)

사용자가 등록한
불변의 고정 반복 일정
(예: 매주 수요일 10:30
수업).

```
{  
  "classTime": "10:30.0",  
  "targetArrivalTime": "3:30.0",  
  "transportMode": "erax"  
}
```

외부 변수 개입
(날씨, 교통 혼잡도)



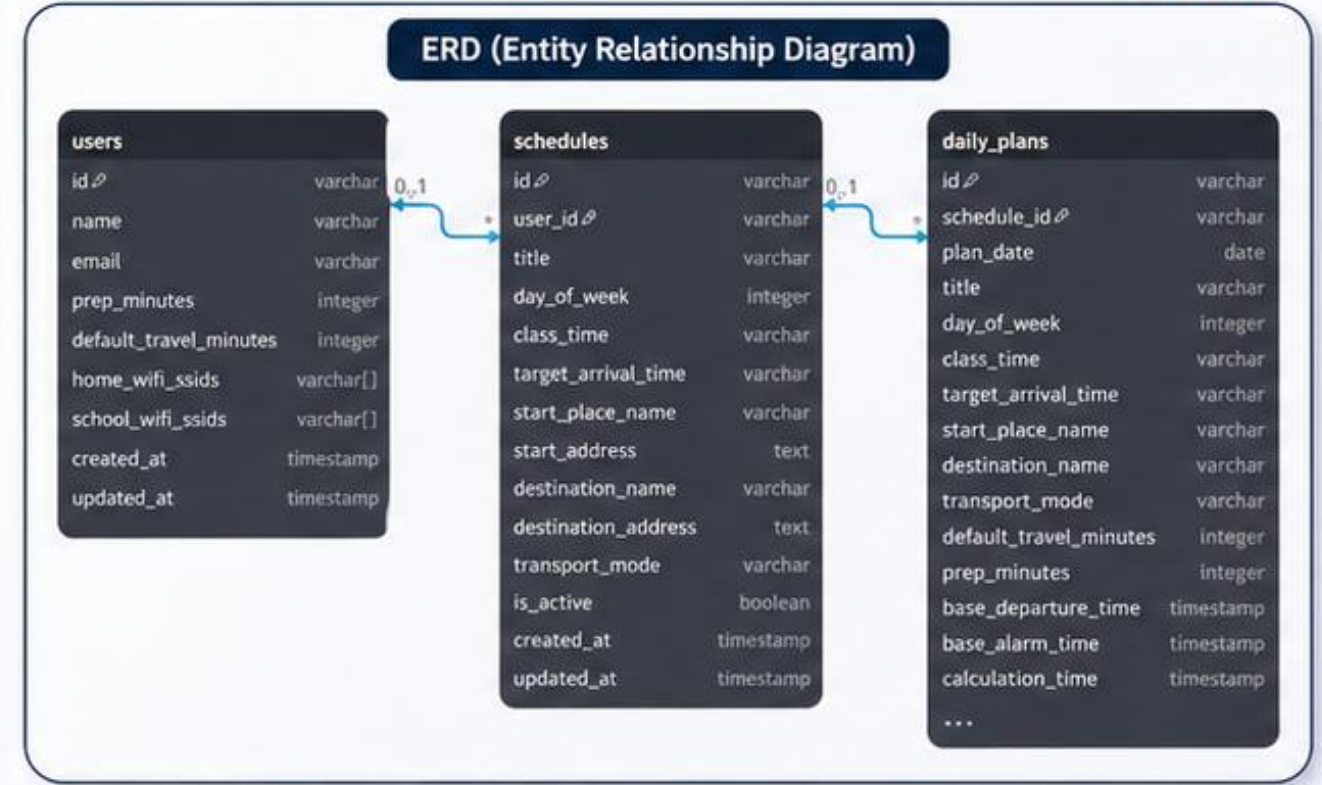
`DailyPlans` (동적 결과물)

템플릿이 외부 변수를
만나 최종 도출된 출발
지침(Instance)으로
생성.

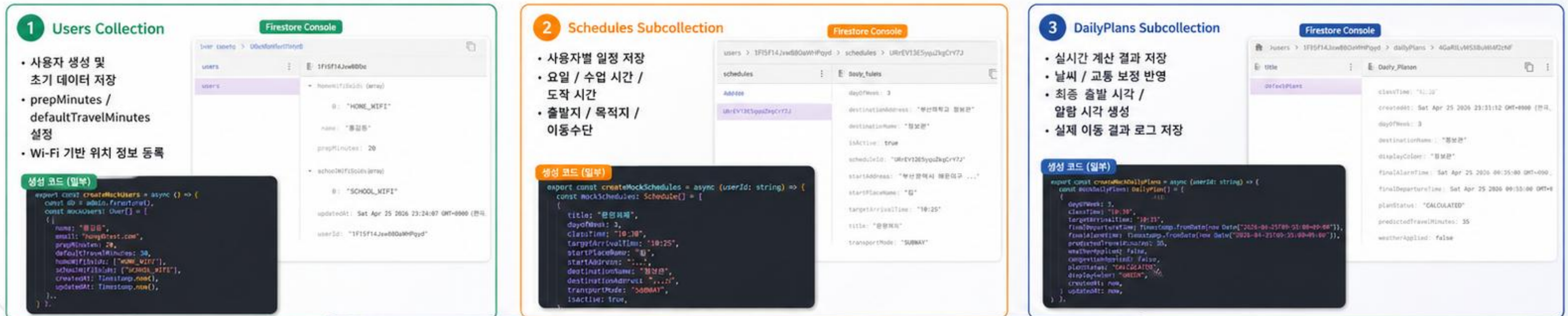
```
"DailyPlans": {  
  "finalDepartureTime": "8:37:09",  
  "finalAlarmTime": "8:32:00",  
  "weather-adjusted_factors": "nalDepartureTime",  
  "weatherApplied": true,  
  "weatherApplied": false,  
  "weatherApplied": true  
  ...  
}
```


1. DB 구조 설계 (ERD 중심) 데이터 흐름 기반 DB 구조 설계

사용자 맞춤 시간 계산을 위한 데이터 구조 설계



2. Firestore 구조 & 데이터 생성 흐름 Firestore 기반 데이터 저장 및 생성 프로세스



실시간 데이터와 누적 데이터를 함께 저장하여 개인화된 시간 예측 알고리즘의 기반을 구축했습니다.

Hardware Setup: 스탠드얼론(Standalone) 동적 알람 기기



무선 네트워크 환경 구축 완료

PC 연결 없이 **단독**으로 **WiFi 통신** 및 **데이터 수신** 가능.

디스플레이 모듈 통합

모바일 앱의 **4단계 상태**(Green~Gray)를 **물리적 알람 시계 화면**에도 동일하게 표출하기 위한 **하드웨어 기반** 완성.

Real-time Sync: Python 앱 기반 DB 통신 테스트 완료

```
jeje9893@nagaja2026: ~$ python3 firebase_test.py
python3: can't open file '/home/jeje9893/firebase_test.py': [Errno 2] No such
file or directory
jeje9893@nagaja2026: ~$ cd nagaja
jeje9893@nagaja2026: ~/nagaja $ ls
firebase_test.py  requirements.txt  testKey.json
github_token.txt  serviceAccountKey.json  venv
jeje9893@nagaja2026: ~/nagaja $ source venv/bin/activate
(venv) jeje9893@nagaja2026: ~/nagaja $ python3 firebase_test.py

쓰기 완료 : ('message': '라즈베리파이에서 연결 성공', 'timestamp':
DatetimeWithMicroseconds(2026, 4, 26, 3, 16, 14, 879000, tzinfo=datetime.timezone.
utc))
(venv) jeje9893@nagaja2026: ~/nagaja $
```

```
...~$ nano firebase_test.py
File ~/nagaja/venv/lib/python3.13/site-packages/google/api_core/retry/retry_base.py, line 218, in _retry_error_helper
raise final_exc from source_exc
File ~/nagaja/venv/lib/python3.13/site-packages/google/api_core/retry/retry_unary.py, line 147, in _retry_target
result = target()
File ~/nagaja/venv/lib/python3.13/site-packages/google/api_core/timeout.py, line 12, in func_with_timeout
return func(*args, **kwargs)
File ~/nagaja/venv/lib/python3.13/site-packages/google/api_core/grpc_helpers.py, line 57, in error_from_rpc_error
raise exceptions.from_grpc_error(exc) from exc
google.api_core.exceptions.NotFound: 404: The database (default) does not exist for project nagaja-4646db. Please visit https://console.cloud.google.com/datastore/setup?project=nagaja-4646db to add a Cloud Datastore or Cloud Firestore database.
(venv) jeje9893@nagaja2026: ~/nagaja $ nano firebase_test.py

(venv) jeje9893@nagaja2026: ~/nagaja $ python3 firebase_test.py

쓰기 완료
읽기 완료 : ('message': '라즈베리파이에서 연결 성공', 'timestamp': DatetimeWithMicroseconds(2026, 4, 26, 3, 16, 14, 879000, tzinfo=datetime.timezone.utc))
(venv) jeje9893@nagaja2026: ~/nagaja $ nano firebase_test.py

(venv) jeje9893@nagaja2026: ~/nagaja $
```



Firestore

Cloud Firestore > 데이터베이스 (default) > Firestore를 사용하기 위한 핵심 개념에 대해 Gemini에게 물어보기

데이터베이스 > 데이터 > test > hello

(default)	test	hello
+ 컬렉션 시작	+ 문서 추가	+ 필드 추가
project01	hello >	+ 필드 추가
test >		message: "라즈베리파이에서 연결 성공"
		timestamp: 2026년 4월 26일 오전 12시 4분 32초 UTC+9



Firestore 통신 스크립트

Python 기반으로 Cloud Firestore 초기화 및 인증(testKey.json)을 성공적으로 완료했습니다.

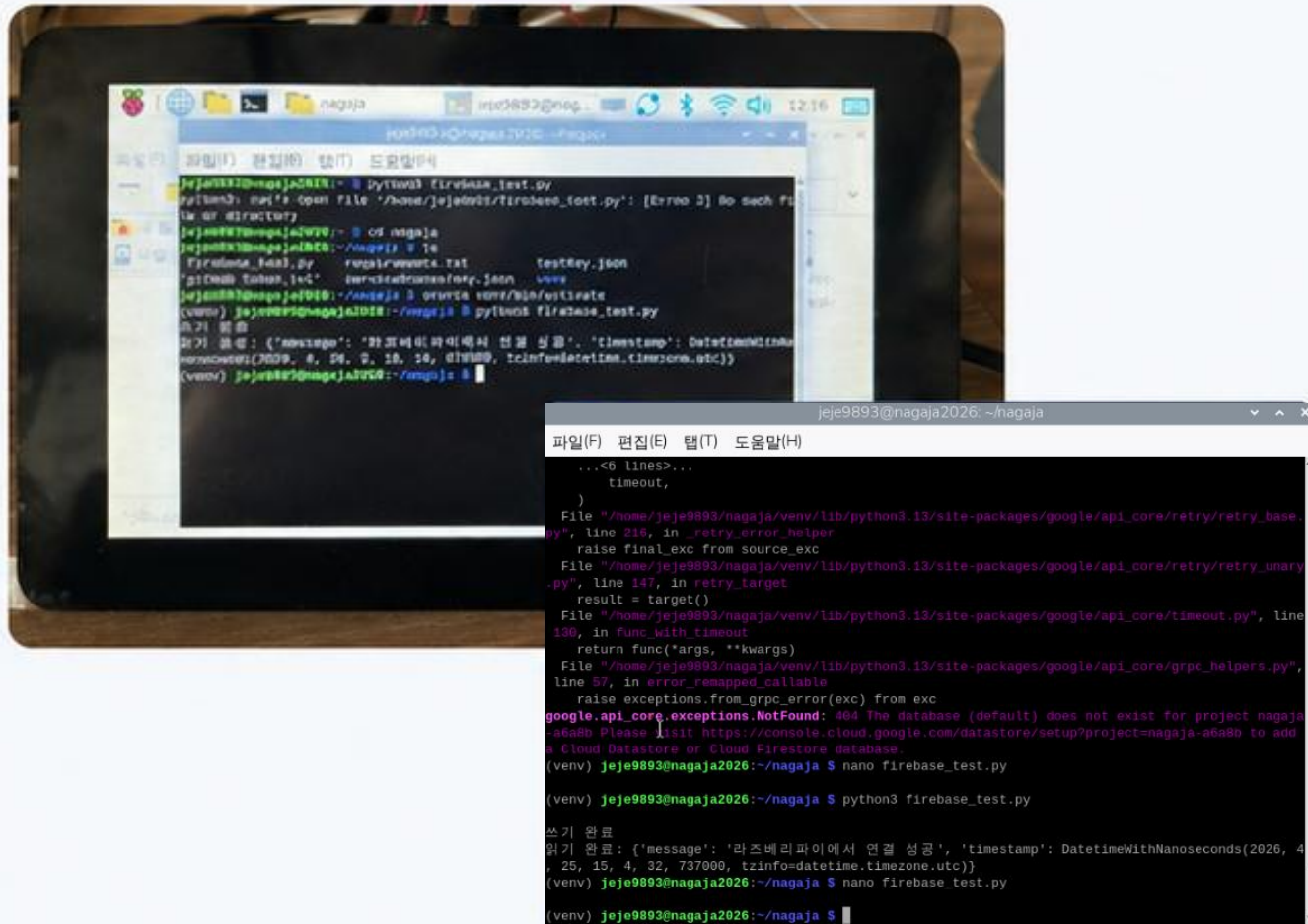


Read / Write 검증

라즈베리파이에서 전송한 메시지와 SERVER_TIMESTAMP가 클라우드 DB에 실시간으로 저장·조회되는 것을 확인하여 양방향 데이터 파이프라인이 정상적으로 동작함을 검증했습니다.

Real-time Sync: 실제 하드웨어 구동 테스트 완료

라즈베리파이 터미널 실행



Python 앱 실행 및 로그 확인

라즈베리파이에서 Python 앱을 실행하여
Firebase 연동 및 메시지 전송 로그를 확인했습니다.

라즈베리파이 실제 하드웨어



실제 하드웨어 구동

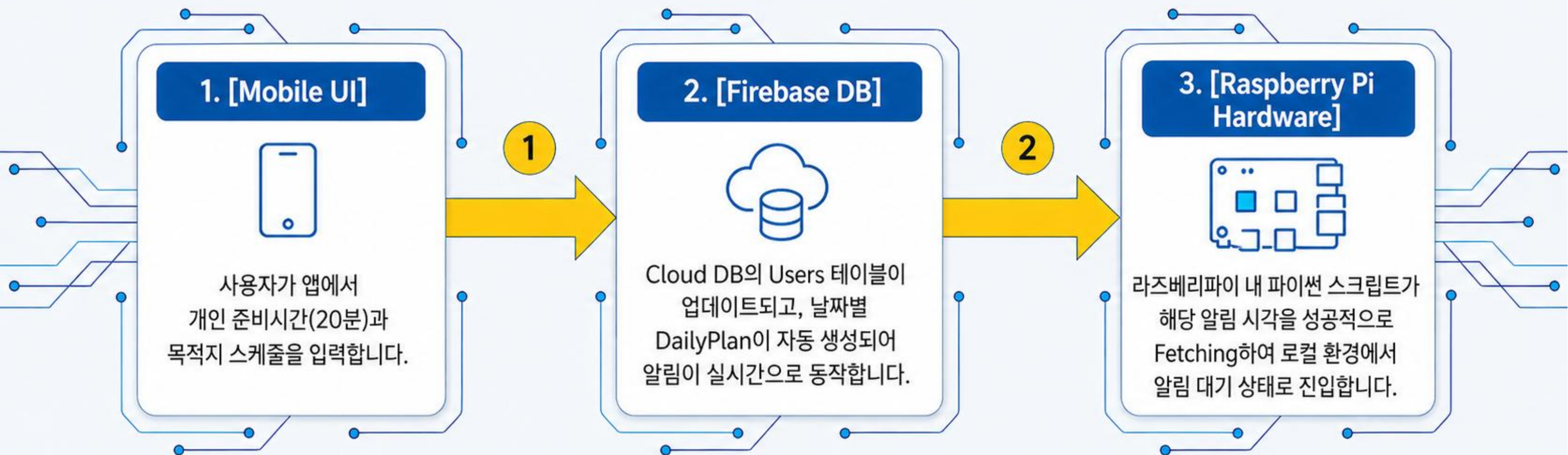
라즈베리파이에 연결된 디스플레이 및 주변 장치가 정상적으로 구동되는 것을 확인했습니다.



라즈베리파이 하드웨어에서 Python 앱 실행 및 Firebase 연동을 통해 실시간 데이터 전송 · 저장이 정상적으로 동작함을 확인했습니다.

End-to-End Milestone:

통합 모의 데이터 파이프라인 동작 확인



**결론: 단절된 개별 컴포넌트 개발을 넘어,
모든 시스템이 연결되는 End-to-End 데이터 파이프라인이 정상적으로 동작함을 확인했습니다.**

Next Steps:

완성형 동적 대시보드를 향한 남은 70% 과제

✓
현재 시점
(30%)

[외부 데이터 연동]

기상청/OpenWeather API를 활용해
날씨에 따른 이동 시간 패널티를 적용하고,
Kakao Maps 기반 대중교통 지연 정보를
실시간 반영



[자기 개선 알고리즘 고도화]

누적된 캘린더 출발 데이터를 기반으로
개인별 이동 시간 및 준비 시간을
자동 보정하고,
수학적 모델 정교화

[하드웨어 완성]

RGB LED 및 진동 모터 제어 코드 구현
알람 미루기 불가 액션 추가
3D 프린팅 외관 케이스 제작

